



# Hafta 4 — Sipariş Motoru (State Machine) & Admin Ürün Yönetimi

Proje: QR Menü Otomasyonu | Tarih: 8 Mart 2026 | Harcanan Süre: ~20 Saat

## 1. Proje Dosya Yapısı (Hafta 4 Güncellemeleri)

```
QRMenu/
├── src/
│   ├── QRMenu.Core/
│   │   ├── Entities/
│   │   │   └── Siparis.cs        ← RowVersion default düzeltmesi (Array.Empty)
│   │   ├── Enums/
│   │   │   └── SiparisDurum.cs  ← 9-state enum (Sepette..İade)
│   │   └── Interfaces/
│   │       └── ISiparisService.cs ← [YENİ] Sipariş iş kurallarının kontratı
│   ├── QRMenu.Data/
│   │   └── Services/
│   │       └── SiparisService.cs ← [YENİ] State Machine + Transaction + Concurrent
│   └── QRMenu.Web/
│       ├── Program.cs            ← DI: ISiparisService kaydı eklendi
│       ├── Controllers/
│       │   ├── SiparisController.cs ← [YENİ] 5 endpoint (oluştur/detay/güncelle/1
│       │   └── AdminController.cs   ← [+15 Endpoint] Kategori/Ürün/Opsiyon CRUD +
│       ├── ViewModels/
│       │   └── UrunYonetimViewModel.cs ← [YENİ] Kategori + Ürün + Opsiyon form mod
│       ├── Views/
│       │   ├── Admin/
│       │   │   └── Urunler.cshtml    ← [YENİ] Tam CRUD sayfası (3 modal + JS)
│       │   ├── Sepet/
│       │   │   └── Index.cshtml      ← "Siparişi Gönder" butonu aktif hale getirildi
│       │   └── Shared/
│       │       └── _AdminLayout.cshtml ← Sidebar: Ürünler linki aktif edildi
│       └── wwwroot/
│           ├── css/
│           │   └── admin-urunler.css ← [YENİ] Ürün yönetim sayfası stili
│           ├── uploads/
│           │   └── urunler/          ← [YENİ] Ürün fotoğrafları dizini
└── tests/
```

## 2. Planlanan vs. Gerçekleşen (Haftalık Hedefler)

| Madde                                 | Durum           | Açıklama                                                                       |
|---------------------------------------|-----------------|--------------------------------------------------------------------------------|
| Sipariş Motoru<br>(State Machine)     | ✓<br>Tamamlandı | 9 durumlu state machine ile sipariş yaşam döngüsü yönetimi.                    |
| Transaction ile<br>Atomik Sipariş     | ✓<br>Tamamlandı | Sepet → Sipariş dönüşümü tek transaction içinde.<br>Provider-aware tasarım.    |
| RowVersion<br>Concurrency<br>Kontrolü | ✓<br>Tamamlandı | Eş zamanlı durum güncellemelerinde çakışma tespiti ve kullanıcı bildirimi.     |
| Sipariş Unit Testleri                 | ✓<br>Tamamlandı | 28 unit test: oluşturma, state geçişleri, iptal, sorgulama senaryoları.        |
| Sepet UI → Sipariş<br>Gönder Butonu   | ✓<br>Tamamlandı | Sepet sayfasından AJAX ile sipariş oluşturma + onay ekranı.                    |
| <b>Admin Kategori<br/>CRUD</b>        | 🚀 Bonus         | Kategori ekleme, düzenleme, silme — chip tabanlı UI.                           |
| <b>Admin Ürün CRUD<br/>+ Fotoğraf</b> | 🚀 Bonus         | Ürün ekleme/güncelleme/silme/toggle + fotoğraf upload (max 2MB, jpg/png/webp). |
| <b>Admin Opsiyon<br/>CRUD</b>         | 🚀 Bonus         | Ürüne opsiyon ekleme/silme — modal içi dinamik yönetim.                        |

## 3. Sipariş Motoru — State Machine Mimarisi

### 3.1 Durum Akış Diyagramı

```
[Sepette] → [Onaylandı] → [Hazırlanıyor] → [Hazır] → [Teslim Edildi] 0 1
2 3 4 | | | | | → [Kısmi Ödendi] → [Tam Ödendi] | | | 5 6 | | | | →
[iptal] → [İptal] → [İade] ← 7 7 8
```

### 3.2 State Machine Kural Tablosu

| Mevcut Durum  | Enum | İzin Verilen Geçişler          |
|---------------|------|--------------------------------|
| Sepette       | 0    | Onaylandı                      |
| Onaylandı     | 1    | Hazırlanıyor, İptal            |
| Hazırlanıyor  | 2    | Hazır, İptal                   |
| Hazır         | 3    | Teslim Edildi                  |
| Teslim Edildi | 4    | Kısmi Ödendi, Tam Ödendi, İade |
| Kısmi Ödendi  | 5    | Tam Ödendi                     |
| Tam Ödendi    | 6    | İade                           |
| İptal         | 7    | — Son durum (terminal) —       |
| İade          | 8    | — Son durum (terminal) —       |

### 3.3 State Machine Kod Implementasyonu

```
private static readonly Dictionary<SiparisDurum, SiparisDurum[]> _gecisKurallari =
{
    [SiparisDurum.Sepette] = new[] { SiparisDurum.Onaylandi },
    [SiparisDurum.Onaylandi] = new[] { SiparisDurum.Hazirlaniyor, SiparisDurum.
    [SiparisDurum.Hazirlaniyor] = new[] { SiparisDurum.Hazir, SiparisDurum.Iptal }
    [SiparisDurum.Hazir] = new[] { SiparisDurum.TeslimEdildi },
    [SiparisDurum.TeslimEdildi] = new[] { SiparisDurum.KismiOdendi, SiparisDurum.T
    [SiparisDurum.KismiOdendi] = new[] { SiparisDurum.TamOdendi },
    [SiparisDurum.TamOdendi] = new[] { SiparisDurum.Iade },
    [SiparisDurum.Iptal] = Array.Empty<SiparisDurum>(),
    [SiparisDurum.Iade] = Array.Empty<SiparisDurum>()
};
```

## 4. Transaction Yönetimi & Concurrency Kontrolü

#### Atomik Transaction

- Sepet → Sipariş dönüşümü tek transaction içinde
- SepetDetay → SiparisDetay kopyalama
- Sepet temizleme (ürünler + toplam sıfırlama)

#### Concurrency (RowVersion)

- Sipariş entity'sinde `byte[] RowVersion`
- EF Core `[Timestamp]` ile otomatik kontrol
- `DbUpdateConcurrencyException` yakalama

- Hata durumunda otomatik rollback

- Kullanıcıya "sayfa yenile" mesajı

## 4.1 Provider-Aware Transaction Stratejisi

**Sorun:** InMemory database provider'ı transaction desteklemiyor.

**Çözüm:** `ProviderName` kontrolü ile runtime'da provider tespiti. PostgreSQL (Npgsql) için `ExecutionStrategy` + `BeginTransactionAsync` kullanılırken, InMemory testlerde transaction atlanıyor. Bu şekilde aynı servis kodu hem production hem test ortamında sorunsuz çalışıyor.

```
var supportsTransactions = _context.Database.ProviderName
    != "Microsoft.EntityFrameworkCore.InMemory";

if (supportsTransactions)
{
    var strategy = _context.Database.CreateExecutionStrategy();
    return await strategy.ExecuteAsync(
        () => ExecuteSiparisOlusturAsync(sepetId, notlar, true));
}
return await ExecuteSiparisOlusturAsync(sepetId, notlar, false);
```

## 4.2 Sipariş Oluşturma Akış Şeması



`COMMIT`

Transaction başarılı → Sipariş döndür

## 5. Sipariş Controller — API Endpoint'leri

| HTTP | Route                   | Açıklama                                                                     |
|------|-------------------------|------------------------------------------------------------------------------|
| POST | /siparis/olustur        | Sepetteki ürünleri siparişe çevirir. Body: { SepetId, Notlar? }              |
| GET  | /siparis/{id}           | Sipariş detayını getirir (detay satırları + ürün bilgisi dahil).             |
| POST | /siparis/durum-guncelle | State Machine kurallarına uygun durum geçişi. Body: { SiparisId, YeniDurum } |
| GET  | /siparis/siparislerim   | Aktif oturumun tüm siparişlerini listeler.                                   |
| POST | /siparis iptal/{id}     | Siparişi iptal eder (sadece Onaylandı/Hazırlanıyor durumlarından).           |

## 6. Admin Panel — Ürün & Kategori & Opsiyon Yönetimi

### 6.1 Admin Controller Yeni Endpoint'ler (15 adet)

| HTTP                | Route                         | Açıklama                                   |
|---------------------|-------------------------------|--------------------------------------------|
| 📁 Kategori Yönetimi |                               |                                            |
| GET                 | /admin/urunler                | Ürün yönetim sayfası (View)                |
| GET                 | /admin/kategoriler            | Tüm kategorileri JSON olarak döndürür      |
| POST                | /admin/kategori-ekle          | Yeni kategori oluşturur                    |
| POST                | /admin/kategori-guncelle/{id} | Kategori bilgilerini günceller             |
| POST                | /admin/kategori-sil/{id}      | Kategori siler (ürünü varsa engeller)      |
| 📁 Ürün Yönetimi     |                               |                                            |
| GET                 | /admin/urun-detay/{id}        | Ürün detayı (opsiyonlar dahil) JSON        |
| POST                | /admin/urun-ekle              | Yeni ürün ekler ( FromForm — fotoğraf ile) |

| HTTP                | Route                     | Açıklama                                                |
|---------------------|---------------------------|---------------------------------------------------------|
| POST                | /admin/urun-guncelle/{id} | Ürün bilgilerini günceller (fotoğraf değişimi dahil)    |
| POST                | /admin/urun-sil/{id}      | Ürünü ve fotoğrafını siler                              |
| POST                | /admin/urun-toggle/{id}   | Ürün aktif/pasif durumunu değiştirir                    |
| ⚙️ Opsiyon Yönetimi |                           |                                                         |
| POST                | /admin/opsiyon-ekle       | Ürüne opsiyon ekler (yeni oluşturur veya mevcut bağlar) |
| POST                | /admin/opsiyon-sil/{id}   | Ürün-opsiyon bağlantısını kaldırır                      |

## 6.2 Fotoğraf Upload Güvenliği

### Güvenlik Kontrolleri:

- **Dosya boyutu:** Maksimum 2MB ( `MaxFileSize = 2 * 1024 * 1024` )
- **Uzantı kontrolü:** Sadece `.jpg` , `.jpeg` , `.png` , `.webp` kabul edilir
- **GUID dosya adı:** Orijinal dosya adı kullanılmaz — `Guid.NewGuid()` + `ext`
- **Eski dosya temizleme:** Fotoğraf güncellendiğinde veya ürün silindiğinde eski dosya disk'ten silinir
- **Depolama:** `/wwwroot/uploads/urunler/` dizinine kaydedilir

## 6.3 Admin UI Bileşenleri

### 📁 Kategori Yönetimi

- Chip tabanlı kategori listesi
- Her chip'te düzenle/sil ikonları
- Yeni kategori + inline düzenleme
- Ürün bağımlılık kontrolü (silme engeli)

### 🖼️ Ürün Tablosu

- Fotoğraf + ad + kategori + fiyat kolonları
- Aktif/pasif toggle switch
- Kategori dropdown filtresi
- İsme göre arama

### 📷 Fotoğraf Upload

- Dosya seçimi ile anlık önizleme
- FormData ile AJAX gönderim
- 2MB ve format kısıtlaması
- Güncelleme sırasında eski dosya silme

### ⚙️ Opsiyon Yönetimi

- Ürün bazlı modal ile opsiyon listesi
- Ad + Grup + Ek Fiyat ile yeni opsiyon
- Tek tıkla opsiyon kaldırma
- Mükerrer kontrolü

## 7. Düzeltilen Kritik Hatalar

| Hata                                   | Kategori     | Çözüm                                                                                                                                                               |
|----------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| InMemory Transaction Hatası            | Testing      | InMemory DB transaction desteklemediği için <code>ConfigureWarnings</code> ile <code>TransactionIgnoredWarning</code> bastırıldı + provider-aware service tasarımı. |
| RowVersion null Hatası (InMemory)      | Testing      | InMemory provider <code>RowVersion</code> otomatik üretmiyor — default <code>null!</code> yerine <code>Array.Empty&lt;byte&gt;()</code> atandı.                     |
| ExecutionStrategy InMemory Uyumsuzluğu | Architecture | <code>CreateExecutionStrategy().ExecuteAsync()</code> InMemory'de çalışmıyor — <code>Database.ProviderName</code> kontrol edilerek runtime'da dallanma yapıldı.     |

## 8. Test ve Doğrulama (59/59 PASSED)

| Test Grubu                               | Test Sayısı | Durum          |
|------------------------------------------|-------------|----------------|
| Ürün Servisi Testleri (Hafta 1)          | 7           | ✓ Geçti        |
| Token/Oturum Testleri (Hafta 2)          | 9           | ✓ Geçti        |
| Sepet ve Hesaplama Testleri (Hafta 3)    | 15          | ✓ Geçti        |
| <b>Sipariş Motoru Testleri (Hafta 4)</b> | <b>28</b>   | <b>✓ Geçti</b> |

### 8.1 Sipariş Testleri Detayı (28 Test)

| #                 | Test                                                         | Sonuç |
|-------------------|--------------------------------------------------------------|-------|
| Sipariş Oluşturma |                                                              |       |
| 1                 | Başarılı sipariş oluşturma — sepet verisi doğru kopyalanmalı | ✓     |
| 2                 | Boş sepet ile sipariş — hata fırlatmalı                      | ✓     |
| 3                 | Geçersiz sepet ID — hata fırlatmalı                          | ✓     |
| 4                 | Sipariş notları doğru kaydedilmeli                           | ✓     |

### State Machine — Geçerli Geçişler (10 Senaryo)

|    |                              |   |
|----|------------------------------|---|
| 5  | Sepette → Onaylandı          | ✓ |
| 6  | Onaylandı → Hazırlanıyor     | ✓ |
| 7  | Onaylandı → İptal            | ✓ |
| 8  | Hazırlanıyor → Hazır         | ✓ |
| 9  | Hazırlanıyor → İptal         | ✓ |
| 10 | Hazır → Teslim Edildi        | ✓ |
| 11 | Teslim Edildi → Kısmi Ödendi | ✓ |
| 12 | Teslim Edildi → Tam Ödendi   | ✓ |
| 13 | Kısmi Ödendi → Tam Ödendi    | ✓ |
| 14 | Tam Ödendi → İade            | ✓ |

### State Machine — Geçersiz Geçişler (6 Senaryo)

|    |                                         |   |
|----|-----------------------------------------|---|
| 15 | Sepette → Hazırlanıyor (atladı!) — hata | ✓ |
| 16 | Onaylandı → Teslim Edildi — hata        | ✓ |
| 17 | Hazır → Onaylandı (geri!) — hata        | ✓ |
| 18 | İptal → Onaylandı (terminal!) — hata    | ✓ |
| 19 | İade → Onaylandı (terminal!) — hata     | ✓ |
| 20 | Olmayan sipariş ID — hata               | ✓ |

### İptal & Sorgulama

|    |                                                                       |   |
|----|-----------------------------------------------------------------------|---|
| 21 | Onaylandı durumundan iptal                                            | ✓ |
| 22 | Teslim Edildi durumundan iptal — hata (yalnız Onaylandı/Hazırlanıyor) | ✓ |
| 23 | Sipariş detayı getirme (detay satırları + ürün)                       | ✓ |
| 24 | Masa bazlı sipariş listesi (iptal hariç)                              | ✓ |
| 25 | Oturum bazlı sipariş listesi                                          | ✓ |

### Geçerli Geçiş Kuralları



26 Onaylandı'nın geçerli geçişleri: Hazırlanıyor, İptal



27 İptal'in geçerli geçişleri: boş dizi (terminal)



28 Teslim Edildi → İade geçişi



```
> dotnet test tests\QRMenu.Tests\QRMenu.Tests.csproj
Başarılı! - Başarısız: 0, Başarılı: 59, Atlanan: 0, Toplam: 59 ✓
```

## 9. Bu Haftada Kullanılan Teknolojiler & Desenler

| Teknoloji / Desen            | Kullanım Amacı                                                                 |
|------------------------------|--------------------------------------------------------------------------------|
| State Machine Pattern        | Sipariş yaşam döngüsü — kurala aykırı geçişleri engelleme                      |
| TransactionScope             | Sepet → Sipariş dönüşümünü atomik yapma (ya hep ya hiç)                        |
| RowVersion /<br>Timestamp    | Eş zamanlı durum güncellemelerinde çakışma kontrolü (Optimistic Concurrency)   |
| ExecutionStrategy            | Npgsql retry logic ile transient hata toleransı                                |
| IFormFile + GUID             | Güvenli fotoğraf upload (dosya adı sanitization + boyut/format kontrolü)       |
| AJAX (Fetch API)             | Sayfa yenilemeden kategori/ürün/opsiyon CRUD işlemleri                         |
| FormData                     | Fotoğraf ile birlikte form verisi gönderimi (multipart/form-data)              |
| Bootstrap 5 Modals           | 3 farklı modal: Kategori düzenleme, Ürün ekleme/düzenleme, Opsiyon yönetimi    |
| xUnit +<br>InlineData/Theory | Parametrik test senaryoları ile 10 geçerli + 6 geçersiz state transition testi |

## 10. Gelecek Hafta Planı (Hafta 5)

### Hafta 5 — Mutfak & Garson Paneli:

- Mutfak ekranı (Kanban tarzı sipariş kartları): Onaylandı → Hazırlanıyor → Hazır
- Garson paneli: Masa bazlı sipariş listesi + durum güncelleme
- SignalR entegrasyonu (gerçek zamanlı bildirim — sipariş gelince mutfakta otomatik gösterim)

- Admin panel sipariş yönetimi sayfası